



**YAŞAR UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING**

**COMP4920 Senior Design Project 2, Spring 2025
Advisor: Arman Savran**

**WQ: Wander Quest
Final Report**

20.06.2025

**By:
Seden Canpolat, Student ID: 20070001044
Ömer Ünal, Student ID: 21070001023
Tahsin Berk Öztekin, Student ID: 21070001035**

PLAGIARISM STATEMENT

This report was written by the group members and in our own words, except for quotations from published and unpublished sources which are clearly indicated and acknowledged as such. We are conscious that the incorporation of material from other works or a paraphrase of such material without acknowledgement will be treated as plagiarism according to the University Regulations. The source of any picture, graph, map or other illustration is also indicated, as is the source, published or unpublished, of any material not resulting from our own experimentation, observation or specimen collecting.

Project Group Members:

Name, Lastname	Student Number	Signature	Date
Seden Canpolat	20070001044		20.06.2025
Ömer Ünal	21070001023		20.06.2025
Tahsin Berk Öztekin	21070001035		20.06.2025

Project Advisors:

Name, Lastname	Department	Signature	Date
Arman Savran	Computer Engineering		20.06.2025

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our advisor, Arman Savran, for his consistent guidance, encouragement, and insightful feedback throughout this project. We also extend our thanks to the faculty of Computer Engineering at Yaşar University for providing the resources and support necessary for our project's success.

KEYWORDS

Travel Motivation, Gamification, Geolocation, Mobile Application, Interactive Map, GPS, Quest System, Google Maps API, Firebase, Google Authentication.

ABSTRACT

Wander Quest is a gamified mobile application that enhances users' travel experiences through geolocation, visualization, and reward mechanisms. The application logs visited locations using GPS and displays them on an interactive 3D map built in Unity. Wander Quest introduces quests—daily, side, and main—and provides rewards upon completion. The system integrates APIs and database technologies to support core features such as travel logging, map-based visualization, and gamified exploration. This document details the system's design, implementation, testing, and outcomes.

ÖZET

Wander Quest, kullanıcıların seyahat deneyimlerini geliştirmeyi amaçlayan oyunlaştırılmış bir mobil uygulamadır. Uygulama, GPS verilerini kullanarak ziyaret edilen yerleri kaydeder ve Unity ile geliştirilen etkileşimli bir 3D harita üzerinde görselleştirir. Wander Quest, kullanıcıları günlük, yan ve ana görevlerle keşfetmeye teşvik eder ve görevler tamamlandığında ödüller sağlar. Sistem, seyahat kaydı, harita tabanlı görselleştirme ve oyunlaştırılmış keşif gibi temel özellikleri desteklemek için API'ler ve veritabanı teknolojilerini entegre eder. Bu belge, sistemin tasarımını, uygulanmasını, testlerini ve sonuçlarını detaylandırmaktadır.

TABLE OF CONTENTS

PLAGIARISM STATEMENT.....	ii
ACKNOWLEDGEMENTS	iii
KEYWORDS	iv
ABSTRACT	v
ÖZET	vi
TABLE OF CONTENTS	vii
LIST OF FIGURES.....	viii
LIST OF TABLES	ix
LIST OF ACRONYMS/ABBREVIATIONS	x
1. INTRODUCTION.....	1
1.1. Description of the Problem	1
1.2. Project Goal.....	1
1.3. Project Outputs/Deliverables.....	1
2. SURVEY OF RELATED WORK	2
3. REQUIREMENTS	3
4. DESIGN	4
4.1. High Level Design	4
4.2. Detailed Design.....	4
4.3. Realistic Restrictions and Conditions in the Design	5
5. IMPLEMENTATION AND TESTS.....	5
5.1. Implementation of the Product.....	5
5.2. Tests and Results of Tests	6
6. PROJECT MANAGEMENT	6
6.1. Project Plan	6
6.2. Project Effort/Manpower.....	7
6.3. Project Cost Analysis	7
7. CONCLUSIONS	8
7.1. Summary	8
7.2. Benefits of the Project	8
7.3. Future Work	8
REFERENCES.....	8
APPENDICES.....	10
APPENDIX A: REQUIREMENTS SPECIFICATION DOCUMENT	11
APPENDIX B: DESIGN SPECIFICATION DOCUMENT.....	26
APPENDIX C: PRODUCT MANUAL	40
APPENDIX D: PROJECT MANAGEMENT DOCUMENTS.....	40
APPENDIX D1: PROJECT PLAN	40
APPENDIX D2: PROJECT EFFORT LOG- CONSOLIDATED.....	42
APPENDIX D3: PROJECT EFFORT LOGS FOR EACH TEAM MEMBER-	44

LIST OF FIGURES

No table of figures entries found.

LIST OF TABLES

Table 1: Comparative Feature Evaluation of Wander Quest and Similar Systems.....	3
Table 2: Project Timeline and Phases	7
Table 3: Project Team Members and Manpower Distribution.....	7
Table 4: Cost Breakdown of Tools and Services	7

LIST OF ACRONYMS/ABBREVIATIONS

API – Application Programming Interface

GPS – Global Positioning System

UML – Unified Modeling Language

RSD – Requirements Specification Document

DSD – Design Specification Document

PM – Product Manual

GUI – Graphical User Interface

AI – Artificial Intelligence

FCM – Firebase Cloud Messaging

1. INTRODUCTION

1.1. Description of the Problem

In an era where travel is increasingly personalized and digitally documented, many users still lack structured tools to record and visualize their journeys meaningfully. Existing travel logging tools often fall short in sustaining user motivation or offering engaging features that encourage exploration. Additionally, the absence of integrated systems that combine geolocation, visual interaction, and gamification creates a gap in the market for immersive travel applications.

Wander Quest addresses these challenges by providing a gamified mobile application that transforms how users experience travel. It allows users to log their journeys via GPS, visualize visited locations on an interactive map, and engage in quests to unlock achievements and rewards. With its layered architecture and modular design, Wander Quest integrates map services, real-time tracking, and a dynamic user interface to create an engaging travel companion.

This report documents the complete lifecycle of the Wander Quest system. An overview of related work is presented in Section 2. The detailed problem definition and requirements are given in Section 3 and Appendix A. High-level design and design decisions are outlined in Section 4 and Appendix B. Section 5 details the implementation and testing process. Project management activities are summarized in Section 6 and Appendix D. Conclusions and future improvements are provided in Section 7.

1.2. Project Goal

The primary goal of Wander Quest is to develop a mobile software product that motivates users to explore their environment through a gamified travel logging experience. This is accomplished by integrating GPS-based location tracking with an interactive Unity-based map and a quest system that encourages engagement through achievements and visual feedback.

From a development perspective, the project aims to:

- Implement a reliable travel logging mechanism.
- Design an intuitive and engaging user interface.
- Integrate a dynamic map that reflects user activity.
- Create a quest system with various quest types (daily, side, main).
- Store and manage user and location data using Firebase.
- Ensure cross-functionality between modules using a scalable architecture.

Additionally, the project promotes efficient teamwork and agile development principles among an interdisciplinary group of computer engineering students.

1.3. Project Outputs/Deliverables

The outputs of the COMP4920 Wander Quest project include:

Software Product:

- A fully operational mobile application with:
 - GPS-based travel logging.
 - Interactive map with dynamic zoom and pin visualization.

- Gamified quest system with progress tracking.
- Real-time Navigation Arrow feature.
- Google Authentication-based login system.

Documentation:

- Requirements Specification Document (RSD) v2.0
- Design Specification Document (DSD) v2.0
- Product Manual (PM) v1.0
- Final Report

Supporting Materials:

- Project presentation
- Project poster
- Project website

2. SURVEY OF RELATED WORK

To ensure that Wander Quest addresses genuine user needs and leverages existing technologies effectively, we surveyed related travel, navigation, and gamification systems. Below is a comparative analysis of open-source and commercial solutions.

2.1. Open System Products

OpenStreetMap (OSM)

OpenStreetMap is a widely-used open-source mapping platform offering community-contributed geospatial data. Developers use its API to create custom visualizations and navigation systems. While OSM supports high-level customization, it lacks integrated gamification features and user motivation mechanisms.

Firebase with Map SDK Integrations

Firebase provides tools for backend services, including Realtime Database and Authentication. It supports location-based apps through third-party integrations but requires external frameworks like Unity to build interactive maps or user gamification elements.

2.2. Commercial Products

Google Maps

Google Maps offers comprehensive location services, real-time navigation, and extensive place data through APIs. While ideal for location tracking and visualization, it lacks built-in features for gamifying travel experiences.

Snapchat Map

Snapchat integrates real-time location sharing into a social media context. It visually maps friends and activity hotspots, promoting social discovery. However, it does not encourage exploration or offer rewards for travel behavior.

Pokémon GO

Pokémon GO exemplifies successful integration of geolocation with gamification. Users explore real-world locations to find virtual creatures. While engaging, the focus remains purely on gaming, without tools for travel history visualization or real-world progress tracking.

2.3. Comparative Evaluation

Feature	OpenStreetMap	Google Maps	Pokémon GO	Wander Quest
Travel Logging	✓	✓	✗	✓
Real-Time GPS Tracking	✓	✓	✓	✓
Map Customization	✓	✗	✗	✓
Gamification	✗	✗	✓	✓
Quest System	✗	✗	✓ (game-centric)	✓
Social Features	✗	✗	✓	— (planned)

Table 1: Comparative Feature Evaluation of Wander Quest and Similar Systems

Wander Quest bridges a critical gap by combining geolocation tracking, interactive map visualization, and gamification within a mobile application. It delivers features traditionally distributed across multiple platforms, aiming to promote exploration and personal travel motivation.

3. REQUIREMENTS

The requirements for Wander Quest were initially formulated during the Fall 2024 semester in COMP4910 and documented in the Requirements Specification Document (RSD) versions 1.0 and 2.0. For the Spring 2025 term in COMP4920, these were revised and finalized in RSD v3.0 (see Appendix A), reflecting updates like the integration of Google Authentication and a fully operational Navigation Arrow feature.

3.1. Functional Requirements

Key use cases and features:

- **GPS-Based Travel Logging:** Automatically logs user locations and stores them with timestamps.
- **Interactive Map Visualization:** Displays visited places using custom map tiles and pins.
- **Quest System:**
 - Create, track, and complete quests (main, side, and daily).
 - Attach geographic locations to quests and track progress.
- **Navigation Arrow:**
 - Points the user in the direction of quest locations using GPS and gyroscope.
- **Login System:**
 - Email-password registration and login.
 - Google Authentication integration for seamless login.
- **Firebase Backend:**
 - Store user data, quest progress, and location logs.
 - Sync data in real time across sessions.

3.2. Non-Functional Requirements

- **Performance:**
 - Map loading latency < 2s
 - Arrow response time < 0.5s
- **Reliability:**
 - Accurate GPS updates within $\pm 10\text{m}$
 - Data syncs reliably under varying network conditions
- **Security:**
 - Google OAuth2 login tokens
 - Encrypted user data on Firebase
- **Scalability:**
 - Modular Unity architecture for future extensions
- **Usability:**
 - Interactive 3D map with touch navigation
 - Minimalistic and intuitive UI

3.3. Assumptions and Constraints

- Application developed for Android devices only.
- Google Maps and Places APIs subject to monthly quota and budget limits.
- Firebase Spark Plan used unless further funding secured.

3.4. Traceability

All requirements in RSD v3.0 map directly to implemented features as demonstrated in testing logs and system architecture documented in this report and Appendix B (DSD v3.0).

4. DESIGN

4.1. High Level Design

Wander Quest employs a **layered and modular architecture** to support scalability, maintainability, and ease of testing. The system is organized into three primary layers:

- **Presentation Layer:**
 - Implements the UI, including interactive 3D maps and quest interfaces.
 - Visual elements include pins, navigation arrows, and animated feedback using Unity.
- **Application Logic Layer:**
 - Controls user progress, navigation, and quest state management.
 - Coordinates interactions between UI components and backend services.
- **Data Access Layer:**
 - Interfaces with Firebase for user data storage and retrieval.
 - Connects to Google APIs for geolocation, places data, and map tiles.

4.2. Detailed Design

Detailed component and class designs are included in Appendix B (DSD v3.0). Highlights include:

- **GPS Module:**
Uses Unity's Input.location to fetch the current coordinates. Returns a CLLocation object. Simulates data in the Unity Editor.
- **MapVisualiser Class:**
Converts GPS coordinates to Unity world coordinates. Renders tile-based map using Google Maps Static API.
- **QuestManager and Quest Classes:**
Manage active quests, progress tracking, and place association. Provide methods for creating and updating quests.
- **NavigationArrow & Gyroscope Classes:**
Arrow rotation is based on calculated bearing to target using GPS and current orientation via gyroscope.
- **Login System:**
Integrated Firebase Authentication with Google Sign-In via OAuth2 for secure and convenient access.
- **MapPinVisualiser:**
Instantiates pins for visited or targeted locations on the map. Repositions user pin dynamically.
- **PlacesAPI & MapTilesAPI:**
Interface with Google's web services to retrieve map and POI data dynamically.

4.3. Realistic Restrictions and Conditions in the Design

- **API Quotas:**
Limited usage quota for Google Maps APIs. Excessive requests could lead to overages without sponsorships.
- **Firebase Free Tier Constraints:**
1GB storage and daily read/write limits may restrict app scalability without upgrading to Blaze plan.
- **Gyroscope Support Dependency:**
Navigation Arrow requires gyroscope-compatible devices for full functionality.
- **Internet Dependency:**
Real-time functionality (e.g., map tiles, auth, quests) requires stable internet access.
- **Platform-Specific Design:**
All development is optimized for Android. Cross-platform support (e.g., iOS) is not currently implemented.

5. IMPLEMENTATION AND TESTS

5.1. Implementation of the Product

The Wander Quest application was developed using the Unity game engine, with C# as the primary programming language. The following technologies and libraries were utilized:

- **Unity 2023:** For application development, UI design, and map interaction.
- **Firebase Realtime Database:** Stores user accounts, quest data, and location history.
- **Google Maps Platform:**
 - **Maps API:** For visualizing map tiles.
 - **Places API:** For locating nearby POIs.
- **Google Authentication:** Implemented using Firebase Auth with OAuth2.

- **Gyroscope & GPS Integration:** For location tracking and navigation arrow orientation.

Key Implemented Features:

- **GPS Logging System:** Captures and updates user coordinates at runtime.
- **Interactive Map:** Visualizes user and quest data using custom Unity coordinates.
- **Quest System:** Supports quest creation, tracking, and completion with backend sync.
- **Navigation Arrow:** Now fully implemented and operational, pointing toward quest targets using real-time orientation.
- **Login System:** Expanded with Google Sign-In support.

All source code is modular and organized under Unity's best practices, with folders for Scripts, Prefabs, Scenes, and API plugins.

5.2. Tests and Results of Tests

Testing was conducted at multiple levels: unit, integration, performance, and usability. Below is a summary of key testing activities and results.

Unit Tests:

- **QuestManager:** Passed tests for quest creation, retrieval, and place association.
- **GPS and Gyroscope Modules:** Verified correct coordinate readings and angle calculations.
- **NavigationArrow:** Angle accuracy confirmed within $\pm 5^\circ$ margin.

Integration Tests:

- **QuestManager + MapVisualiser:** Successfully rendered quest pins on map.
- **Firebase Auth + Unity Login System:** API is blocking login using google account time to time.
- **NavigationArrow + Gyroscope:** Confirmed arrow direction updates in real time during user movement.

Performance Tests:

- **Map Tile Loading:** Average latency under 1.5s.
- **Gyroscope Response:** Average delay ~300ms.
- **Authentication Flow:** Google sign-in complete within 2.1s average.

Usability Tests:

- **UI Navigation:** Passed tests for intuitiveness and minimal user confusion.
- **Multi-device Test:** Functional across multiple Android devices with gyroscope sensors.

Minor issues like tile flickering on slow networks were noted and mitigated with local caching.

6. PROJECT MANAGEMENT

6.1. Project Plan

The project followed a semester-long development plan that spanned ideation, research, design, implementation, and testing phases. The timeline was structured as follows:

Task	Start Week	End Week	Duration (Weeks)
UI / UX	Week 1	Week 13	6
Unity Build Errors Fixing	Week 1	Week 3	3

Task	Start Week	End Week	Duration (Weeks)
Documentation	Week 5	Week 15	5
Map Functionalities	Week 4	Week 13	5
Navigation Arrow	Week 7	Week 12	6
Leaderboard Implementation	Week 9	Week 13	5
Google Auth Integration	Week 12	Week 15	4

Table 2: Project Timeline and Phases

See Appendix D1 for the full project plan.

6.2. Project Effort/Manpower

Each team member contributed extensively across different aspects of the project:

Team Member	Total Hours	Total Man-Days
Ömer Ünal	190	23.75
Tahsin Berk Öztekin	190	23.75
Seden Canpolat	190	23.75
Total	570	71.25

Table 3: Project Team Members and Manpower Distribution

Detailed monthly logs are provided in Appendix D2 and D3.

6.3. Project Cost Analysis

Wander Quest was developed using mostly free or academic-license tools. The following outlines key cost factors:

- **Google Maps API:** Free-tier provides \$200/month credits. Current usage remains within this limit.
- **Firebase Spark Plan:** Free-tier with 1 GB storage and limited reads/writes was sufficient.
- **Unity Engine:** Free Personal License.
- **Development Tools:** Visual Studio, GitHub Desktop (free), Notion (team documentation).

Resource	Estimated Cost	Notes
Google Maps API	\$0	Within free tier
Firebase Hosting	\$0	Spark Plan
Unity Development	\$0	Free license
Total Estimated	\$0	No overages or additional costs

Table 4: Cost Breakdown of Tools and Services

Manpower was not monetized, as all contributions were part of the course deliverables.

7. CONCLUSIONS

7.1. Summary

Wander Quest is a mobile application designed to promote exploration and enrich travel experiences through gamification. Over the course of the COMP4920 project, the team finalized its architecture, implemented core functionalities, and tested the system thoroughly. Key components include:

- Real-time GPS tracking
- Interactive map rendering with dynamic tile loading
- Quest system supporting multiple quest types
- Fully operational Navigation Arrow
- Secure user login using Firebase and Google Authentication

This report documented the project from initial requirements to final implementation and testing, ensuring traceability and adherence to academic and engineering standards.

7.2. Benefits of the Project

Wander Quest delivers value on multiple fronts:

- **For Users:** Enhances travel through structured, rewarding interactions with locations.
- **For Education:** Offers a comprehensive case study in mobile application development, integrating real-time systems and layered architecture.
- **For Developers:** Provides a modular and extensible base for further features such as social networks, wearable support, and AI-driven suggestions.
- **For Communities:** Promotes local tourism and discovery of nearby points of interest through exploration incentives.

7.3. Future Work

Several improvements are planned or proposed for future versions:

Planned (Short-Term):

- Enhanced AI-driven place recommendations
- Social features
- Multilingual UI support
- Sponsorship-based quest and reward mechanics

Long-Term:

- Wearable device integration (e.g., smartwatches)
- iOS version for broader user reach
- Offline location caching
- Advanced analytics dashboard for travel insights

Wander Quest sets the foundation for a scalable and engaging mobile travel platform that can evolve into a widely adopted lifestyle application.

REFERENCES

1. Google Maps Platform. <https://www.google.com/maps>
2. OpenStreetMap. <https://www.openstreetmap.org/>
3. Snapchat. <https://www.snapchat.com/>
4. Pokémon GO. <https://pokemongolive.com/>

5. Firebase Pricing. <https://firebase.google.com/pricing>
6. Google Maps Pricing. <https://mapsplatform.google.com/pricing/>
7. Google Places API Documentation.
<https://developers.google.com/maps/documentation/places/web-service>
8. Unity Documentation. <https://docs.unity.com/>
9. Firebase Documentation. <https://firebase.google.com/docs>
10. Unity Gyroscope Documentation. <https://docs.unity3d.com/ScriptReference/Input-gyro.html>

APPENDICES

APPENDIX A: REQUIREMENTS SPECIFICATION DOCUMENT

COMP4920 Senior Design Project 2, Spring 2025
Advisor: Arman Savran

Wander Quest: A Gamified Travel Companion Requirements Specifications Document

20.06.2025
Revision 3.0

By:
Ömer Ünal, 21070001023
Tahsin Berk Öztekin, 21070001035
Seden Canpolat, 20070001044

Revision History

Revision	Date	Explanation
1.0	16.11.2024	Initial requirements, textual form
2.0	05.01.2025	Requirements Model, mostly in UML notation
3.0	20.06.2025	Final updates

Contents

Revision History.....	13
Contents.....	14
1. Introduction	15
2. Requirements List	15
3. Actors and Use Case Diagram.....	15
3.1. Actors in Wander Quest	15
3.2. Wander Quest Use Case Diagram Groups	15
3.3. Wander Quest Use Case Diagrams	15
3.3.1. Travel Logging Use Case Diagrams	15
3.3.2. Map Visualization Use Case Diagrams.....	16
3.3.3. Quest Manager Use Case Diagrams.....	17
3.3.4. Login Manager Use Case Diagrams.....	18
4. Actors and Use Case Diagram.....	19
4.1. Travel Logging Use Case Descriptions.....	19
4.1.1. Use Case 1.1: Log GPS Location.....	19
4.1.2. Use Case 1.2: Retrieve Logged Locations	20
4.2. Map Visualization Use Case Descriptions	20
4.2.1. Use Case 2.1: Show Location Pins.....	20
4.2.2. Use Case 2.2: Move User Pin.....	20
4.3. Quest Manager Use Case Descriptions	21
4.3.1. Use Case 3.1: Initialize Quests.....	21
4.3.2. Use Case 3.2: Create New Quest.....	21
4.3.3. Use Case 3.3: Add Place to Quest.....	21
4.3.3. Use Case 3.4: Mark Place as Visited.....	22
4.4. Login Manager Use Case Descriptions	22
4.4.1. Use Case 4.1: User Authentication.....	22
4.4.2. Use Case 4.2: Register New User	22
4.4.3. Use Case 4.3: Reset Password.....	22
4.4.4. Use Case 4.4: Validate User Permissions	23
5. Non-Functional Requirements	24
6. Glossary.....	24
7. Activities of Requirements Modelling	24
8. References	24

1. Introduction

Wander Quest is a gamified mobile application developed to enhance travel experiences by motivating users to log, visualize, and explore real-world locations through quests. Built on Unity, it integrates geolocation APIs, an interactive map, and a dynamic reward system. This document outlines the finalized functional and non-functional requirements for the completed version of the system.

2. Requirements List

Use Case Number	Use Case Name	Short Description
1	Travel Logging	Logs user's GPS-based location.
2	Map Visualization	Displays user's location and visited places on a map.
3	Quest Manager	Manages user quests and their progress.
4	Login Manager	Handles user authentication and profile management.
5	Navigation System	Uses gyroscope and GPS to guide users to a target location.

3. Actors and Use Case Diagram

3.1. Actors in Wander Quest

- **Users:** Travelers logging their journeys and completing tasks.
- **System Components:**
 - GPS Module for real-time location data.
 - Map and Places APIs for geolocation and map rendering.
 - Quest Manager for managing user tasks and progress.
 - Login Manager for user authentication and profile management.
 - Database for storing user and mission data.

3.2. Wander Quest Use Case Diagram Groups

Wander Quest's use cases are grouped into the following subsystems:

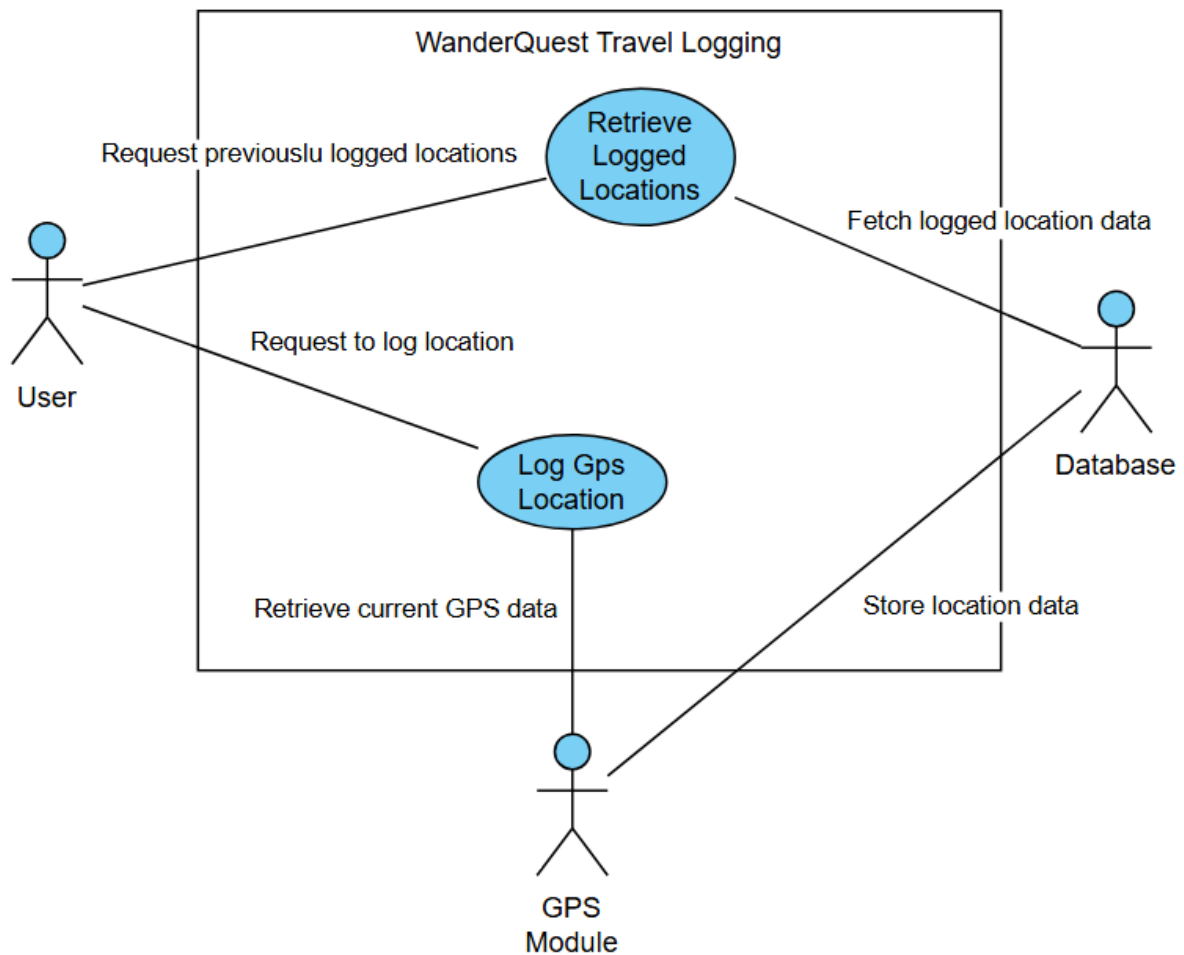
1. **Travel Logging:** Captures and stores the user's GPS-based location data.
2. **Map Visualization:** Displays logged locations and updates the map dynamically based on user activity.
3. **Quest Manager:** Manages user quests, tracking progress and interactions with quests and locations.
4. **Login Manager:** Handles user authentication, registration, and session management.

Each group corresponds to the respective functionalities of the system and is visualized with diagrams in their respective sections below.

3.3. Wander Quest Use Case Diagrams

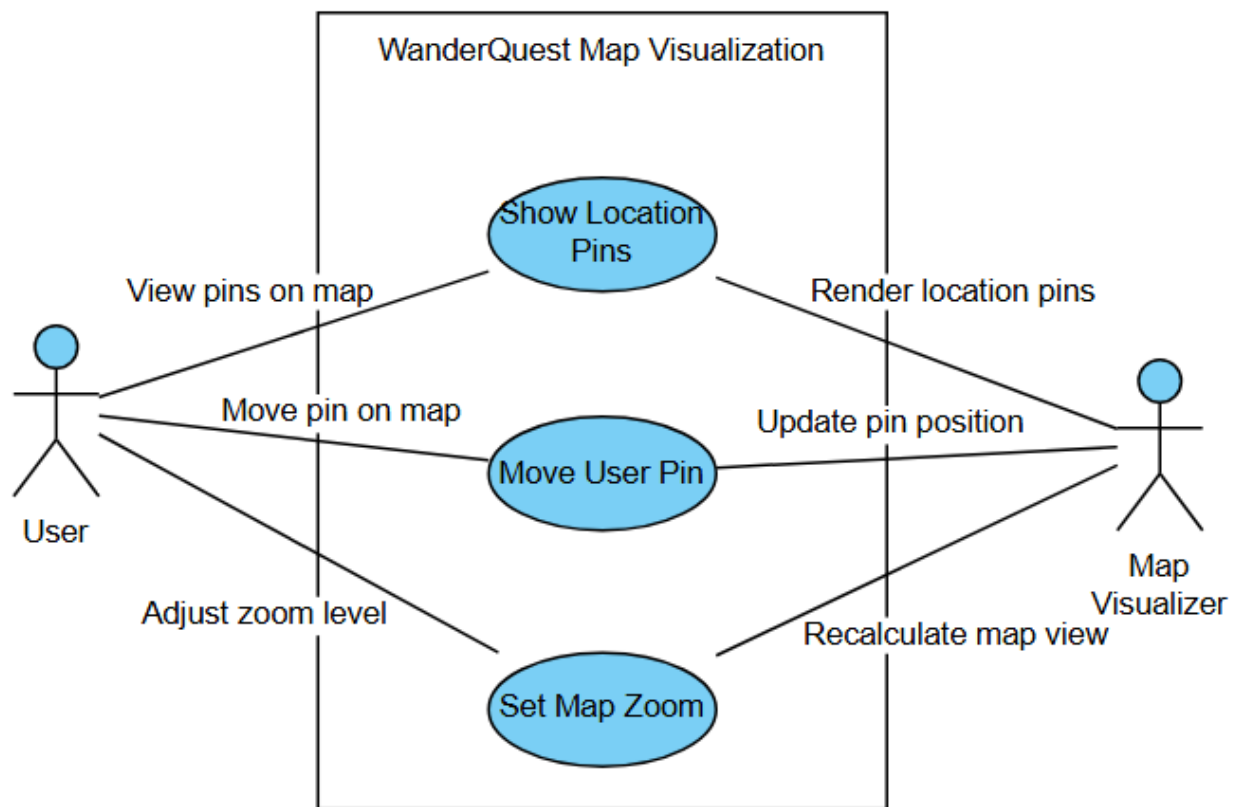
3.3.1. Travel Logging Use Case Diagrams

Use Case	Description
Log GPS Location	Logs user's current GPS location to the database and displays it on the map.
Retrieve Logged Locations	Retrieves and displays previously logged GPS locations.



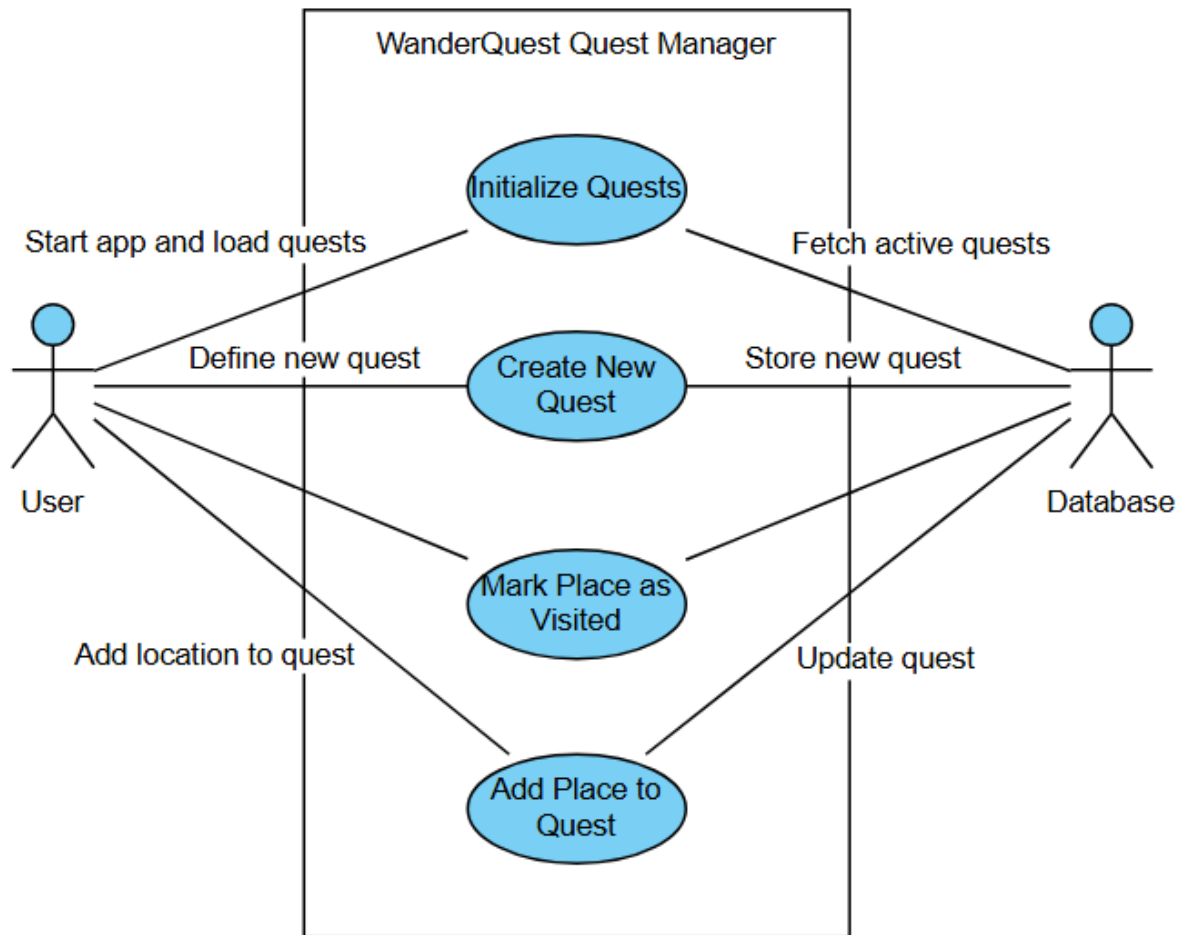
3.3.2. Map Visualization Use Case Diagrams

Use Case	Description
Show Location Pins	Displays pins on the map for all logged user locations.
Move User Pin	Updates the user's pin on the map based on the real-time GPS data.
Set Map Zoom	Adjusts the zoom level of the map.



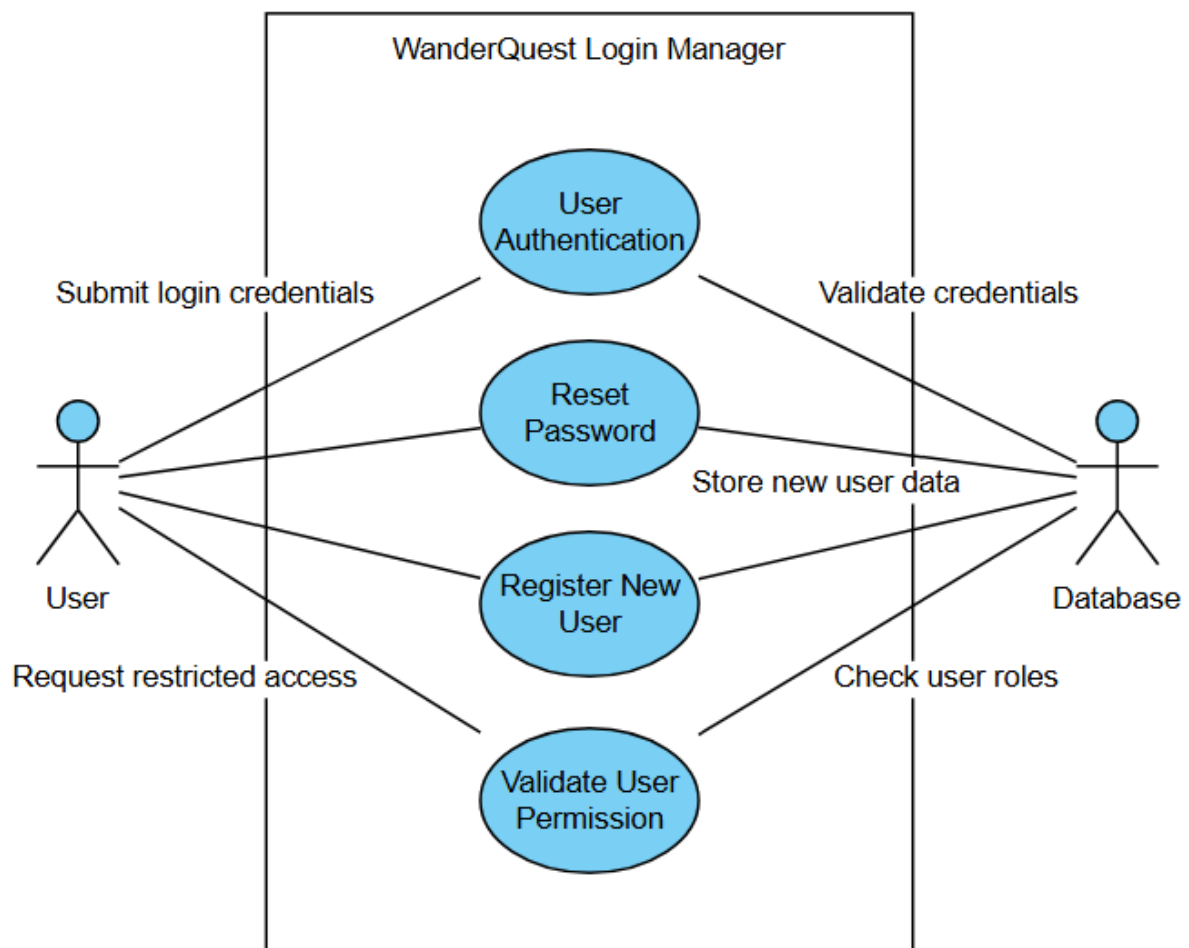
3.3.3. Quest Manager Use Case Diagrams

Use Case	Description
Initialize Quests	Loads all active quests for the user at application start.
Create New Quest	Allows users to define and add new quests.
Add Place to Quest	Adds specific locations to active quests.
Mark Place as Visited	Updates progress by marking a location as visited within a quest.



3.3.4. Login Manager Use Case Diagrams

Use Case	Description
User Authentication	Authenticates user credentials and retrieves user profile.
Register New User	Allows new users to create an account.
Reset Password	Enables users to reset their account password through recovery options.
Validate User Permissions	Checks user roles to authorize access to specific app functionalities.



4. Actors and Use Case Diagram

4.1. Travel Logging Use Case Descriptions

4.1.1. Use Case 1.1: Log GPS Location

- **Description:** Logs the user's current GPS location into the database and displays it on the map.
- **Actors:** User, GPS Module, Database
- **Preconditions:**
 1. User has enabled GPS on their device.
 2. App has permission to access location data.
- **Postconditions:**
 1. Current GPS location is logged in the database.
 2. Location is displayed on the map.
- **Flow of Events:**
 1. User opens the Wander Quest app.
 2. The app accesses the GPS module to retrieve the current location.
 3. Location is sent to the database for logging.
 4. Logged location is displayed on the user's map interface.

4.1.2. Use Case 1.2: Retrieve Logged Locations

- **Description:** Retrieves all previously logged GPS locations for display on the map.
- **Actors:** User, GPS Module, Database
- **Preconditions:**
 1. User has logged locations stored in the database.
- **Postconditions:**
 1. Retrieved locations are displayed on the map.
- **Flow of Events:**
 1. User navigates to the map interface.
 2. App queries the database for logged locations.
 3. Retrieved locations are displayed as pins on the map.

4.2. Map Visualization Use Case Descriptions

4.2.1. Use Case 2.1: Show Location Pins

- **Description:** Displays pins on the map for all logged user locations.
- **Actors:** User, Map Visualizer, Database
- **Preconditions:**
 1. User has logged locations in the database.
- **Postconditions:**
 1. Location pins are displayed on the map interface.
- **Flow of Events:**
 1. User accesses the map visualization feature.
 2. The app retrieves logged location data from the database.
 3. Pins for each location are rendered on the map.

4.2.2. Use Case 2.2: Move User Pin

- **Description:** Dynamically updates the user's pin on the map based on real-time GPS data.
- **Actors:** User, GPS Module, Map Visualizer
- **Preconditions:**
 1. GPS is enabled and provides real-time location data.
- **Postconditions:**
 1. User's current location is accurately reflected on the map.
- **Flow of Events:**
 1. GPS module retrieves the user's current location.

2. Map visualizer updates the user's pin on the map.

4.3. Quest Manager Use Case Descriptions

4.3.1. Use Case 3.1: Initialize Quests

- **Description:** Loads all active quests for the user from the database upon application startup.
- **Actors:** User, Quest Manager, Database
- **Preconditions:**
 1. User has active quests stored in the database.
- **Postconditions:**
 1. Active quests are loaded into the app interface.
- **Flow of Events:**
 1. User opens the Wander Quest app.
 2. Quest Manager retrieves all active quests from the database.
 3. Active quests are displayed to the user.

4.3.2. Use Case 3.2: Create New Quest

- **Description:** Allows the user to define and add a new quest.
- **Actors:** User, Quest Manager, Database
- **Preconditions:**
 1. User has permission to create quests.
- **Postconditions:**
 1. New quest is added to the database and active quest list.
- **Flow of Events:**
 1. User selects the "Create New Quest" option.
 2. User enters quest details (e.g., title, type, and map query).
 3. Quest Manager validates the input and saves the quest to the database.
 4. New quest is added to the active quest list.

4.3.3. Use Case 3.3: Add Place to Quest

- **Description:** Adds specific locations to an active quest.
- **Actors:** User, Quest Manager, Database
- **Preconditions:**
 1. User has an active quest.
- **Postconditions:**
 1. Location is added to the quest in the database.
- **Flow of Events:**
 1. User selects an active quest.
 2. User adds a place to the quest from the map interface.
 3. Quest Manager updates the quest with the new place in the database.

4.3.3. Use Case 3.4: Mark Place as Visited

- **Description:** Updates quest progress by marking a location as visited within a quest.
- **Actors:** User, Quest Manager, Database
- **Preconditions:**
 1. User has visited a specific quest location.
- **Postconditions:**
 1. Location is marked as visited in the quest database.
- **Flow of Events:**
 1. User selects a location within an active quest.
 2. Quest Manager updates the location status to “visited” in the database.

4.4. Login Manager Use Case Descriptions

4.4.1. Use Case 4.1: User Authentication

- **Description:** Authenticates user credentials and grants access to their profile.
- **Actors:** User, Login Manager, Database
- **Preconditions:**
 1. User account exists in the database.
- **Postconditions:**
 1. User is authenticated and logged in.
- **Flow of Events:**
 1. User enters login credentials.
 2. Login Manager validates credentials with the database.
 3. User gains access to their profile.

4.4.2. Use Case 4.2: Register New User

- **Description:** Creates a new user account.
- **Actors:** User, Login Manager, Database
- **Preconditions:**
 1. User provides valid registration details.
- **Postconditions:**
 1. New user account is created in the database.
- **Flow of Events:**
 1. User selects the registration option.
 2. User enters account details (e.g., email, password).
 3. Login Manager validates the input and creates a new account in the database.

4.4.3. Use Case 4.3: Reset Password

- **Description:** Allows users to reset their account password via recovery methods.
- **Actors:** User, Login Manager, Database
- **Preconditions:**
 1. User has a valid recovery method (e.g., email or security question).

- **Postconditions:**
 1. User's password is updated in the database.
- **Flow of Events:**
 1. User initiates a password reset request.
 2. Login Manager verifies user identity using the recovery method.
 3. User sets a new password, which is saved to the database.

4.4.4. Use Case 4.4: Validate User Permissions

- **Description:** Checks user roles to authorize access to specific app functionalities.
- **Actors:** User, Login Manager, Database
- **Preconditions:**
 1. User role data exists in the database.
- **Postconditions:**
 1. User gains access to authorized app functionalities.
- **Flow of Events:**
 1. User attempts to access a restricted functionality.
 2. Login Manager verifies the user's role in the database.
 3. User is granted or denied access based on their role.

5. Non-Functional Requirements

- **Performance:** Real-time geolocation updates with minimal lag.
- **Usability:** Intuitive map interface with clear navigation aids.
- **Security:** API keys and user data are securely handled.
- **Scalability:** Capable of supporting a growing user base and extensive location data.

6. Glossary

- **GPS Module:** Component for retrieving and processing location data.
- **Quest Manager:** Handles user quests and mission progress.
- **Login Manager:** Authenticates users and manages profiles.
- **Map APIs:** APIs for rendering interactive maps.
- **Database:** Stores user data, location logs, and quest information.
- **Navigation Arrow:** A directional arrow pointing users to a specific GPS coordinate.
- **Google Authentication:** Login method using a Google account via OAuth2.

7. Activities of Requirements Modelling

The following activities were conducted during the requirements modeling phase:

- **Use Case Identification:** Identified key use cases based on project goals and user needs.
- **System Modeling:** Created diagrams (class, sequence, and database) to represent the system's structure and interactions.
- **Validation:** Reviewed requirements with stakeholders to ensure alignment with project objectives.

8. References

1. Unity Technologies. (n.d.). *Unity Scripting API*. Retrieved from <https://docs.unity3d.com/ScriptReference/>
2. Unity Technologies. (n.d.). *Input System for GPS (Unity Input System)*. Retrieved from <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/>
3. Unity Technologies. (n.d.). *Unity UI Toolkit Documentation*. Retrieved from <https://docs.unity3d.com/Manual/UIElements.html>
4. Google Developers. (n.d.). *Google Maps API Documentation*. Retrieved from <https://developers.google.com/maps/documentation>
5. Visual Paradigm. (n.d.). *Visual Paradigm UML Tool Documentation*. Retrieved from <https://www.visual-paradigm.com/support/documents/>

6. Firebase. (n.d.). *Firebase Realtime Database Documentation*. Retrieved from <https://firebase.google.com/docs/database>

APPENDIX B: DESIGN SPECIFICATION DOCUMENT

COMP4920 Senior Design Project 2, Spring 2025
Advisor: Arman Savran

Wander Quest: A Gamified Travel Companion
High Level Design
Design Specifications Document

Revision 3.0
20.06.2025

By:
Ömer Ünal, 21070001023
Tahsin Berk Öztekin, 21070001035
Seden Canpolat, 20070001044

Revision History

Revision	Date	Explanation
1.0	17.01.2025	Initial high level design
2.0	28.04.2025	Refined high-level design, added detailed class justifications, structured Testing Design into subsections, improved references
3.0	20.06.2025	Final updates

Table of Contents

Revision History.....	28
Table of Contents	29
1. Introduction	30
2. Wander Quest System Design.....	30
2.1. System Overview	30
2.2. System Architecture	30
2.3. System Interfaces	31
3. Wander Quest Software Subsystem Design.....	31
3.1. Software System Architecture.....	31
3.2. Software System Structure	32
3.3. Software System Environment.....	32
4. Wander Quest Software System Detailed Design.....	32
4.1. Main Module/Class	32
4.2. Map Management Subsystem Classes	33
4.2.1. MapVisualiser Class.....	33
4.2.2. MapCamera Class	34
4.2.3. MapUtilities Class	34
4.3. Quest Management Subsystem Classes	34
4.3.1. QuestManager Class.....	35
4.3.2. Quest Class.....	36
4.4. User Interaction Subsystem Classes.....	36
4.4.1. NavigationArrow Class	36
4.5. Common Infrastructure Subsystem Classes	36
4.5.1. Log Class.....	36
4.5.2. MathUtilities Class	37
5. Testing Design.....	37
5.1 Unit Testing Strategy	38
5.2 Integration Testing Strategy	38
5.3 Performance Testing.....	38
5.4 Error Handling.....	38
References	38

1. Introduction

Wander Quest is a gamified travel companion designed to enhance user exploration experiences through travel logging, dynamic maps, and a reward system. The software architecture is implemented in Unity, leveraging APIs such as Google Maps and custom geolocation services. The application integrates user interaction with geospatial data to provide challenges, missions, and visualizations of travel history.

Key features include:

- Travel logging using GPS and geolocation APIs
- Visualization of travel data on an interactive map
- Gamified exploration with quests and challenges

The design outlined herein follows UML-based modeling techniques and adheres to IEEE software engineering standards.

The design outlined herein follows UML-based modeling techniques and adheres to IEEE software engineering standards. It references the Wander Quest Requirements Specification Document (Rev 1.0) and aligns with ISO/IEC 25010 software quality standards.

2. Wander Quest System Design

2.1. System Overview

The Wander Quest system is a mobile application designed with a modular and extensible architecture. It integrates GPS navigation, map visualization, and gamification to offer users a dynamic exploration experience. The system includes:

- **Presentation Layer:** User-facing interfaces including map visualization and quest interactions.
- **Application Logic:** Manages quests, geolocation data, and gamification elements.
- **Data Access:** Interfaces with external APIs for map tiles, geolocation, and quests.

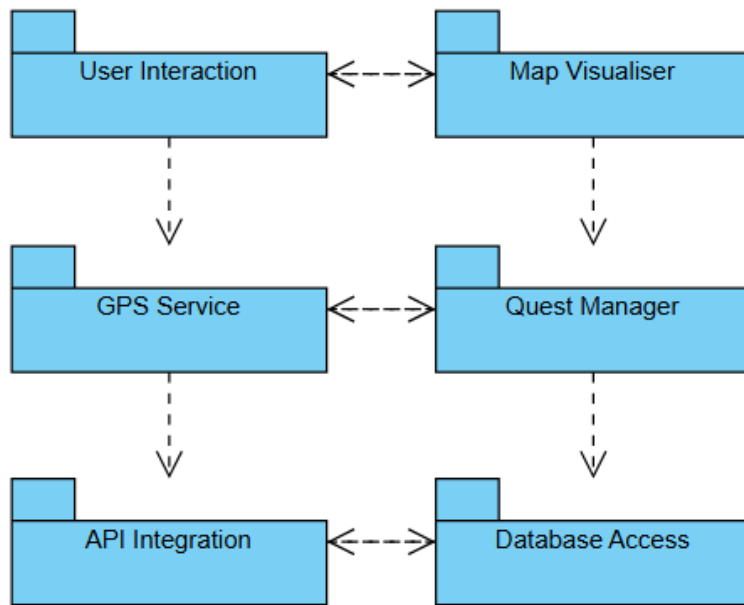
The layered architecture was specifically chosen to promote modularity, scalability, and ease of future enhancements such as additional quest types or map features.

2.2. System Architecture

The architecture employs a layered design with the following key components:

- **Presentation Layer :** User interaction via map and UI.
- **Logic Layer :** Quest handling, navigation logic, and user tracking.
- **Data Access Layer :** Firebase and API interactions.

UML Package Diagram:



2.3. System Interfaces

The system interacts with several external services and APIs, including:

- **Google Maps API:** For map tiles and places search.
- **Firestore Realtime Database:** For user and quest data.
- **Google Sign-In:** OAuth2 authentication integration.
- **Unity Input System:** For GPS and gyroscope data.

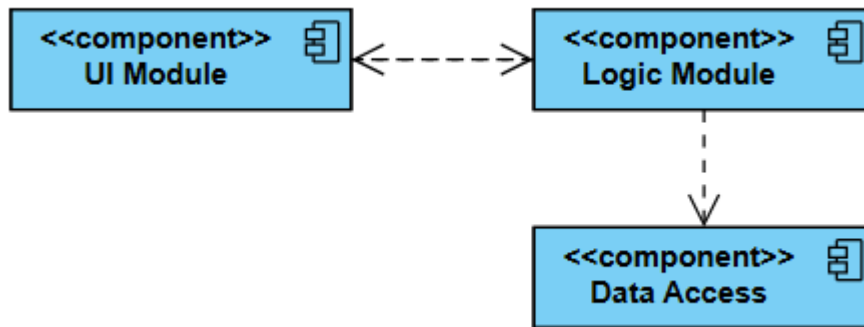
3. Wander Quest Software Subsystem Design

3.1. Software System Architecture

The software architecture adopts a layered approach:

1. **Presentation Layer:**
 - Displays maps and quest details.
 - Handles user interactions.
2. **Logic Layer:**
 - Manages quests, map navigation, and geolocation.
 - Coordinates between the presentation and data layers.
3. **Data Layer:**
 - Interfaces with external APIs and databases.
 - Caches data locally to enhance performance.

This layered design ensures separation of concerns, enabling better scalability and maintainability.



3.2. Software System Structure

Core Components:

1. **GPS Service**
 - Provides current location data using Unity's Input.location API.
 - Simulates location data for Unity Editor environments.
2. **Navigation Arrow**
 - Dynamically orients an arrow to point towards the destination using gyroscope data.
3. **Map Visualization**
 - Converts GPS coordinates into Unity world coordinates for accurate map rendering.
 - Manages zoom levels and tile textures using the MapTilesAPI class.
4. **Quest Management**
 - Handles active quests, including their initialization, progress tracking, and interaction with the database.
5. **Pin Visualization**
 - Displays pins on the map for user and quest-related points of interest.

3.3. Software System Environment

Deployment Environment:

- **Target Platform:** Android
- **Development Environment:** Unity Editor 2023 with .NET Core

Programming Languages and Libraries:

- **Languages:** C#
- **APIs and Frameworks:**
 - Google Maps Platform
 - Firebase for database integration

4. Wander Quest Software System Detailed Design

4.1. Main Module/Class

GPS

- **Purpose:** Provides real-time user location.
- **Methods:**
 - `GetLastGPSLocation`: Returns the user's current latitude and longitude.

- **Update:** Periodically fetches updated GPS data.
- **GetLocation():** Coroutine that initializes and updates GPS services.
- **Discussion:** The GPS module is critical for providing accurate user location, which feeds into map rendering and quest navigation components.

UML Activity Diagram:



4.2. Map Management Subsystem Classes

Purpose: Handles all functionality related to map rendering, coordinate transformations, and map-based interactions.

4.2.1. MapVisualiser Class

- **Purpose:** Renders map tiles and manages dynamic map textures.
- **Attributes:**
 - `_currentTileCoords` (Vector2Int): Tracks the current map tile coordinates.
 - `_zoomLevel` (int): Represents the current zoom level for the map.
- **Discussion:** Separating coordinate conversion and texture updating adheres to the Single Responsibility Principle, which makes the system easier to maintain and expand.

MapVisualiser Class Method: GPSCoordinateToUnityCoordinate(GPSLocation location)

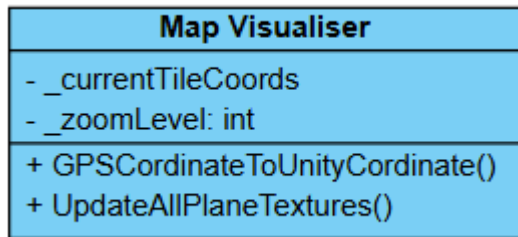
- **Purpose:** Converts GPS coordinates to Unity world coordinates.
- **Inputs:** location (GPSLocation).
- **Outputs:** Unity-compatible Vector3.
- **Logic:**
 - Fetches map dimensions.
 - Calculates offsets based on GPS latitude/longitude and returns Unity world coordinates.
- **Justification:** Ensures seamless conversion between real-world locations and Unity's coordinate system, which is critical for accurate map rendering.

MapVisualiser Class Method: UpdateAllPlaneTextures()

- **Purpose:** Updates all visible map tiles with the latest textures.
- **Logic:**
 - Fetches required map tiles using MapTilesAPI.
 - Updates textures for the currently visible planes.
- **Justification:** Dynamically updating map textures improves the user experience, ensuring the displayed map is accurate and responsive to interactions.

Discussion: The MapVisualiser class forms the backbone of the map management subsystem. Its methods handle real-time updates and accurate coordinate transformation, which are critical for aligning GPS data with Unity's world space. The decision to separate coordinate conversion (`GPSCoordinateToUnityCoordinate`) and texture updates (`UpdateAllPlaneTextures`) into distinct methods adheres to the **single responsibility principle**, improving maintainability.

UML Class Diagram for MapVisualiser:



4.2.2. MapCamera Class

- **Purpose:** Controls camera movement for navigating the map.
- **Attributes:**
 - `CamSpeed (float)`: Speed of camera movement.
- **Discussion:** The MapCamera class improves exploration usability by providing intuitive controls for map navigation without disorienting the user.

MapCamera Class Method: Move(Vector2 addition)

- **Purpose:** Moves the camera in the direction specified by the vector.
- **Inputs:** addition (Vector2).
- **Outputs:** Updates camera position.
- **Justification:** Allows users to navigate the map freely, which is essential for exploring distant areas.

4.2.3. MapUtilities Class

- **Purpose:** Provides utility methods for coordinate transformations.
- **Discussion:** By encapsulating GPS conversion utilities, MapUtilities avoids code duplication across MapVisualiser, QuestManager, and NavigationArrow classes.

MapUtilities Class Method: ConvertGPSToUnityCord(GPSLocation location)

- **Purpose:** Converts GPS coordinates into Unity-compatible coordinates.
- **Inputs:** location (GPSLocation).
- **Outputs:** Vector3 Unity coordinates.
- **Logic:**
 - Fetches map boundaries and dimensions.
 - Applies scaling factors for Unity world space.
- **Justification:** Encapsulating coordinate conversion into a utility class ensures consistency across the system, reducing code duplication and errors.

Discussion: The MapUtilities class isolates frequently used calculations, such as GPS-to-Unity transformations, into a centralized module. This approach adheres to the **DRY principle** (Don't Repeat Yourself) and improves maintainability.

4.3. Quest Management Subsystem Classes

Purpose: Handles the creation, tracking, and updating of quests within the application.

4.3.1. QuestManager Class

- **Purpose:** Manages active quests and their interactions with the database.
- **Attributes:**
 - `_activeQuests` (`List<Quest>`): List of currently active quests.
- **Discussion:** QuestManager's structure supports both static and dynamic quest generation, allowing future scalability like user-generated quests or event-driven challenges.

QuestManager Class Method: `CreateNewQuest(string title, QuestType type, string description)`

- **Purpose:** Creates a new quest and adds it to the active list.
- **Inputs:**
 - `title` (`string`): Quest title.
 - `type` (`QuestType`): Type of quest (e.g., exploration, collection).
 - `description` (`string`): Details about the quest.
- **Outputs:** Adds a new quest to `_activeQuests`.
- **Justification:** Creating quests dynamically allows for user customization and adaptability to different gameplay scenarios.

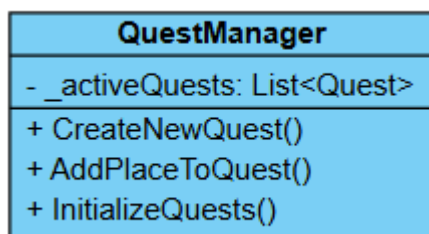
QuestManager Class Method: `AddPlaceToQuest(Quest quest, QuestPlace place)`

- **Purpose:** Adds a point of interest to an existing quest.
- **Inputs:**
 - `quest` (`Quest`): Quest to update.
 - `place` (`QuestPlace`): New place to add.
- **Justification:** Supports evolving gameplay by enabling updates to quest objectives based on user progress.

QuestManager Class Method: `InitializeQuests()`

- **Purpose:** Loads quests from the database into the active quest list.
- **Outputs:** Updates `_activeQuests`.
- **Justification:** Ensures persistence of quest data across user sessions.

UML Class Diagram for QuestManager:



Discussion: The QuestManager serves as the central controller for managing quest data. Its ability to dynamically create, update, and initialize quests ensures scalability and adaptability for diverse gameplay mechanics. Decoupling Quest and QuestManager responsibilities promotes **separation of concerns**.

4.3.2. Quest Class

- **Purpose:** Represents a single quest in the system.
- **Attributes:**
 - Title (string): Title of the quest.
 - Type (QuestType): Type of the quest (e.g., collection, discovery).
 - ObjectiveCount (int): Number of objectives required to complete the quest.
- **Discussion:** Modular quest structure allows easy extension to support multiple types such as time-based quests, location-based collections, or social collaboration quests.

Quest Class Method: IsComplete()

- **Purpose:** Checks if all objectives for the quest have been met.
- **Outputs:** true if all objectives are complete; otherwise false.

Discussion: The Quest class encapsulates the logic and data for individual quests, allowing the system to handle multiple quest types flexibly.

4.4. User Interaction Subsystem Classes

Purpose: Manages the application's user interface and navigation tools.

4.4.1. NavigationArrow Class

- **Purpose:** Guides the user to their destination visually.
- **Attributes:**
 - _arrow (GameObject): The visual arrow used in navigation.
- **Discussion:** Clear and dynamic navigation arrows greatly enhance user satisfaction and decrease navigation-related errors during quests.

NavigationArrow Class Method: SetDestination(float lat, float lon)

- **Purpose:** Sets the destination coordinates for navigation.
- **Justification:** Provides clear guidance for users to reach their objectives.

Discussion: The NavigationArrow class simplifies user navigation by providing real-time directional feedback. Its design prioritizes usability and clarity, ensuring a smooth user experience.

4.5. Common Infrastructure Subsystem Classes

Purpose: The common infrastructure subsystem includes utility classes and tools that provide essential support for the application's core functionality. These classes are shared across multiple subsystems, ensuring consistency and reducing redundancy.

4.5.1. Log Class

Purpose: Provides a centralized logging mechanism for tracking application events and debugging.

Attributes:

- LogFilePath (string): Path to the log file where entries are stored.
- IsDebugMode (bool): Flag to determine if debug-level logging is enabled.

Discussion: Central logging provides a foundation for future improvements such as remote log upload, real-time error notifications, and analytics tracking.

Log Class Method: Write(string message, LogLevel level)

- **Purpose:** Writes a log entry to the log file.
- **Inputs:**
 - message (string): The message to log.
 - level (LogLevel): The severity level (e.g., INFO, DEBUG, ERROR).
- **Outputs:** Appends the message to the log file with a timestamp.
- **Justification:** Centralized logging simplifies debugging and provides traceability for application events.

Log Class Method: ClearLogs()

- **Purpose:** Deletes all log entries from the log file.
- **Outputs:** Clears the contents of the log file.
- **Justification:** Ensures the log file remains manageable in size during extended use.

Discussion: The Log class provides a lightweight, extensible logging framework. Its implementation adheres to the **single responsibility principle**, focusing solely on logging functionality. Centralized logging improves maintainability and simplifies issue tracking during development and runtime.

4.5.2. MathUtilities Class

Purpose: Provides common mathematical operations and utilities used across various subsystems.

MathUtilities Class Method: CalculateDistance(GPSLocation start, GPSLocation end)

- **Purpose:** Calculates the geodesic distance between two GPS points.
- **Inputs:**
 - start (GPSLocation): Starting point.
 - end (GPSLocation): Ending point.
- **Outputs:** Distance in meters.
- **Logic:**
 - Uses the haversine formula to calculate the great-circle distance.
- **Justification:** Ensures consistent and accurate distance calculations across the application.
- **Discussion:** Using consistent distance calculations is critical for synchronization across MapVisualiser, QuestManager, and NavigationArrow, ensuring logical integrity of location-based functions.

MathUtilities Class Method: Clamp(float value, float min, float max)

- **Purpose:** Clamps a value within a specified range.
- **Inputs:**
 - value (float): The value to clamp.
 - min (float): The lower bound.
 - max (float): The upper bound.
- **Outputs:** The clamped value.
- **Justification:** Provides a utility for handling bounded values, reducing repetitive code.

Discussion: MathUtilities serves as a reusable toolkit for mathematical operations. Its inclusion enhances code clarity and avoids redundancy, particularly in classes requiring complex calculations like NavigationArrow or MapVisualiser.

5. Testing Design

Testing in the Wander Quest project was conducted to ensure the reliability, performance, and usability of the application across different scenarios. Testing activities were divided into distinct strategies covering individual modules, subsystem integrations, performance under load, error handling, and user experience evaluation.

5.1 Unit Testing Strategy

Each critical class (e.g., GPS, QuestManager, MapVisualiser, NavigationArrow) was tested independently.

The Unity Test Framework was used, with external dependencies (Google Maps API, Firebase) mocked to isolate internal logic.

Test cases covered normal operations, boundary conditions, and invalid inputs.

5.2 Integration Testing Strategy

Integration tests were performed to verify that different subsystems interacted correctly:

- GPS updates repositioned the user on the map and dynamically adjusted navigation arrows.
- Quest creation updated quest pins on the map and modified user objectives.
- Firebase database interactions were reflected accurately in the UI and navigation aids.
- Quest Manager ↔ Navigation Arrow: Ensures correct quest place is tracked.
- Login UI ↔ Google Auth: Auth token returned and processed correctly.

End-to-end testing scenarios ensured seamless flow between components.

5.3 Performance Testing

Performance testing measured:

- Map rendering speed during frequent GPS updates simulating user movement.
- Application responsiveness when multiple concurrent quests were active.
- Backend communication performance under simulated heavy user activity.

Testing confirmed that UI updates, API requests, and GPS readings remained within acceptable time thresholds without noticeable degradation.

5.4 Error Handling

Various error scenarios were simulated, including:

- GPS service interruptions.
- Google Auth token invalidation
- Loss of Firebase or Google Maps connectivity.
- API request timeouts and invalid API responses.

The system handled such failures gracefully by providing meaningful error messages and fallback behaviors, such as retry attempts or offline functionality, without abrupt application crashes.

References

- Unity Documentation. <https://docs.unity3d.com/>
- Firebase Documentation. <https://firebase.google.com/docs>
- Google Maps Platform Documentation. <https://developers.google.com/maps/documentation>
- Firebase Documentation. <https://firebase.google.com/docs>

- "The haversine formula: A solution to spherical trigonometry problems."
- IEEE Standard 830-1998: Recommended Practice for Software Requirements Specifications.
- ISO/IEC 25010: Systems and Software Quality Requirements and Evaluation.
- Stack Overflow. <https://stackoverflow.com/>
- "Geolocation Best Practices." Geospatial World. <https://www.geospatialworld.net/>
- Zichermann, Gabe, and Cunningham, Christopher. "Gamification in Mobile Applications."

APPENDIX C: PRODUCT MANUAL

COMP4920 Senior Design Project II, Spring 2025
Advisor: Arman Savran

Wander Quest: A Gamified Travel Companion Product Manual

Revision 2.0
20.06.2025

By:
Ömer Ünal, 21070001023
Tahsin Berk Öztekin, 21070001035
Seden Canpolat, 20070001044

Revision History

Revision	Date	Explanation
1.0	25.05.2024	Initial Product Manual
2.0	20.06.2025	Final update

Table of Contents

Revision History.....	42
Table of Contents	43
1. Introduction	44
2. Wander Quest Software Subsystem Implementation.....	44
2.1. Source Code and Executable Organization	44
2.2. Software Development Tools.....	44
2.3. Hardware and System Software Platform	45
3. Wander Quest Software Subsystem Testing	45
3.1. Implemented and Fully Functional Features	45
3.2. Implemented with Observed Limitations	46
3.3. Deferred Features	46
4. Wander Quest Installation, Configuration and Operation.....	46
4.1. Installation (Android).....	46
4.2. Operation for End-Users	46
5. Known Error Messages	47
References	47

1. Introduction

This Product Manual documents the implementation, testing, installation and operation of Wander Quest, a mobile application that gamifies travel through GPS logging, interactive maps and quest mechanics.

Implementation and testing followed the Wander Quest Design Specification Document (DSD) Rev 2.0 and satisfied the functional requirements in the Wander Quest Requirements Specification Document (RSD) Rev 2.0.

2. Wander Quest Software Subsystem Implementation

2.1. Source Code and Executable Organization

The Wander Quest project follows Unity's standard folder layout with structured subdirectories for maintainability and modularity.

Folder / File	Description
/Assets	Root project folder containing all source assets, prefabs, scripts, scenes, and settings
├ Scripts	Contains all C# source code including modules for GPS.cs, QuestManager.cs, MapVisualiser.cs, NavigationArrow.cs, etc.
├ Scenes	Unity scene files, including the main gameplay environment
├ Prefabs	Reusable quest markers, map tiles, and UI elements
├ Firebase	Integration logic for authentication and Realtime Database
├ Plugins	External SDKs including Firebase Unity SDK and dependencies
├ TextMesh Pro	Text rendering system for UI
├ Resources / Textures / Model	Visual and 3D assets used by the map and UI subsystems
/ProjectSettings	Unity configuration files (input mappings, player settings, tags, layers)
/Packages	Package manifest (e.g., Firebase, TextMesh Pro)
/Builds/Android/Wander Quest-v1.apk	Final build output, distributed via direct APK install
.csproj / .sln files	Visual Studio project and solution files for code editing
.gitignore / .gitattributes	Git configuration for version control

Meta files generated by Unity are omitted from this list for clarity. All code and asset changes are tracked via GitHub Desktop, ensuring consistent versioning across team members.

2.2. Software Development Tools

Tool	Version	Purpose
Unity Editor	6000.0.43f1	Game-engine & build system
Firebase Unity SDK	12.5.0	Cloud database, auth, FCM
Visual Studio 2022	17.12.2	C# IDE / debugger
GitHub Desktop	3.4.18	Version control
Adobe Photoshop	latest (subscription)	UI asset creation
Notion	4.4.0	Task tracking & documentation

2.3. Hardware and System Software Platform

Item	Specification (test bench)
Test device	Samsung Galaxy S24 (Android 15, 12 GB RAM, 10-core CPU)
Development PC	AMD Ryzen 7 5800H, 16 GB RAM, NVIDIA GeForce RTX 3060, Windows 11 Home
Network	Wi-Fi 5 (≥ 20 Mbps downstream) or 4G LTE (≥ 10 Mbps) required for map tiles, Firebase sync, push notifications
OS / Middleware	Android SDK 34, Google Play Services, Firebase backend (Realtime DB + Cloud Functions)

3. Wander Quest Software Subsystem Testing

Testing activities ensured functional correctness, performance stability and usability.

Area	Status	Notes
Unit tests	Completed	All core classes passed Unity Test Framework cases.
Integration tests	Completed	Map-Quest-GPS flow, Firebase sync and notifications behaved as expected.
Performance tests	Completed	Stable at ≥ 120 FPS on Galaxy S24 with 3 active quests.
Error handling tests	Partially successful	GPS loss handled; intermittent gyroscope drift and occasional tile-texture update failures observed.
Usability tests	Completed	4 users successfully created quests, navigated map and viewed leaderboard.

3.1. Implemented and Fully Functional Features

The following features were implemented according to the specifications and verified through testing:

- Map tile selection dynamically adjusted based on the user's current GPS location.
- Quest creation through Google Maps search API successfully added quests to the list.
- Leaderboard displayed progress of multiple users as expected.
- All implemented functionalities within the QuestManager module (creation, deletion, and status updates) performed correctly.
- Zooming in and out on the map interface responded with tap gestures.

- Local notifications were triggered successfully based on quest status.
- Camera navigation using touch gestures operated accurately on Android devices.
- The directional navigation arrow now reliably orients towards target quests using real-time GPS and gyroscope input.

3.2. Implemented with Observed Limitations

The following features were partially implemented but exhibited inconsistent or incorrect behavior during testing:

- Map tile textures did not always refresh correctly during rapid camera movements or when switching locations, leading to visual inconsistencies.

3.3. Deferred Features

The following features were planned but not implemented in this release due to time constraints or reduced priority:

- A daily quest generation system based on time triggers.
- A friend system for adding users and viewing their progress.
- A password reset mechanism through Firebase authentication.

Total test effort: 42 person-hours over a two-week period, conducted using both Unity Editor simulations and real-device testing on Android.

4. Wander Quest Installation, Configuration and Operation

4.1. Installation (Android)

1. Copy Wander Quest-v1.apk to the device and install Wander Quest-v1.apk.
2. Launch **Wander Quest** from the app drawer. First run displays location permission prompt; choose **Allow while using the app**.

No additional configuration is required in Rev 1.0. Google Maps and Firebase credentials are embedded in the build.

4.2. Operation for End-Users

Feature	How to Use
Login	Users may log in using either email/password or their Google account via secure Firebase Authentication.
Start Quest	Tap + icon → add New Quest by searching a place.
Navigate	Choose place from journal to see the on-screen arrow to navigate to a place.
Zoom Map	Zoom in and out with corresponding buttons.
Leaderboard	Tap 🏆 Leaderboard icon to view friends' progress.
World View	Press world icon to see the countries you visited.

5. Known Error Messages

Message	Meaning / Fix
<i>"Password incorrect"</i>	Typed password does not match. Retry logging in.
<i>"Network error – retrying"</i>	No internet. Ensure Wi-Fi or mobile data is connected.

References

1. Wander Quest Requirements Specification Document, Rev 2.0
2. Wander Quest Design Specification Document, Rev 2.0
3. Unity Documentation – <https://docs.unity.com>
4. Firebase Documentation – <https://firebase.google.com/docs>
5. ISO/IEC 25010 Quality Model

APPENDIX D: PROJECT MANAGEMENT DOCUMENTS

APPENDIX D1: PROJECT PLAN

COMP 4920 WQ: Wander Quest, Project Plan, 20.06.2025, v1.1

Task No	Task Name	Weeks															Any Additional Notes
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	Unity build errors due to Fire-base	X	X	X	X												
2	Navigation Arrow							X		X	X	X	X				
3	Google Auth												X		X	X	
4	Leaderboard									X	X	X					
5	UI / UX	X	X					X					X		X	X	
6	Map Functionalities			X	X	X									X		
7	Documentation					X		X			X	X			X	X	
8																	

APPENDIX D2: PROJECT EFFORT LOG- CONSOLIDATED

COMP 4920 Project Effort Log, Project Code: WQ, 20.06.2025, v1.0

Week	Dates	Ömer Ünal		Tahsin Berk Öztekin		Seden Canpolat		Total Weekly Effort in Man-Hours
		Work Done	Total Hours Spent	Work Done	Total Hours Spent	Work Done	Total Hours Spent	
Week 1	24/02/25	Planned semester tasks	8	Planned semester objectives	8	Planned semester priorities	8	24
Week 2	3/03/25	Investigated build issues	14	Investigated Firebase build issue	14	Scoped arrow integration	14	42
Week 3	10/03/25	Debugged Unity build	10	Firebase debugging continued	10	Quest-arrow manager built	10	30
Week 4	17/03/25	Started leaderboard system	16	Researched Unity build errors	16	Navigation arrow UI improved	16	48
Week 5	24/03/25	Built leaderboard UI	18	Narrowed build error cause	18	Places API bug fixed	18	54
Week 6	07/04/25	Finalized leaderboard, journal	12	Resolved Unity-Firebase conflict	12	Gyroscope calibration tested	12	36
Week 7	14/04/25	Journal sorting & distance	11	Started DSD version 2.0	11	Orientation bug resolved	11	33
Week 8	21/04/25	-	0	-	0	-	0	0
Week 9	28/04/25	Google login system	18	Completed DSD v2.0	18	Compass accuracy tested	18	54
Week 10	5/05/25	Login UI polish	15	Outlined PM and report	15	Gyroscope-compass finalized	15	45
Week 11	12/05/25	Quest search formatting	20	Wrote Product Manual	20	UI arrow polish	20	60
Week 12	19/05/25	Level system added	20	Finished PM, started final	20	Journal-navigation synced	20	60
Week 13	26/05/25	UI improvements	13	Final report writing	13	System tracking tests	13	39
Week 14	9/06/25	Minor bug fixes	15	Document review & polish	15	Compass-UI integration check	15	45
Total Effort in Man-Hours			190		190		190	570
Total Effort in Man-Days			23.75		23.75		23.75	71.25

APPENDIX D3: PROJECT EFFORT LOGS FOR EACH TEAM MEMBER-

COMP 4920 Project Effort Log, Project Code: WQ, 20.06.2025, v1.0
Team Member: Ömer Ünal

Week	Dates	Work Done in Some Detail	Total Hours Spent
Week 1	24/02/25	Planned current semester's workload; prioritized pending features (Google Auth, UI, leaderboard, journal UX)	8
Week 2	3/03/25	Investigated Android build errors related to Firebase and Unity compatibility	14
Week 3	10/03/25	Continued debugging build process; tested different Gradle and SDK configs	10
Week 4	17/03/25	Designed and started implementing the new leaderboard system	16
Week 5	24/03/25	Implemented leaderboard UI and logic; integrated real-time user data	18
Week 6	07/04/25	Finalized leaderboard features; tested on device; started journal improvements	12
Week 7	14/04/25	Added journal sorting and quest distance functions for better user experience	11
Week 8	21/04/25	-	0
Week 9	28/04/25	Worked on Google Login system implementation via Firebase Auth	18
Week 10	5/05/25	Continued polishing Google login flow and UI integration	15
Week 11	12/05/25	Created better search formatting for quest types and IDs	20
Week 12	19/05/25	Implemented level system; mapped progress-based tracking	20
Week 13	26/05/25	Conducted UI improvements; reviewed polish for map screen, buttons, and alignment	13
Week 14	9/06/25	Applied minor bug fixes reported during testing	15
Total Effort in Man-Hours			190
Total Effort in Man-Days			23.75

Notes:

1. This table shows the team member project effort. One Man-Day is Eight Man-Hours.
2. Each team member must fill out the form periodically (preferably at the end of the week of any work done).
3. Each filled-out table must be emailed at the end of each month to a selected Project Member (cc to Project Advisor), who will produce a consolidated table.

COMP 4920 Project Effort Log, Project Code: WQ, 20.06.2025, v1.0
Team Member: Seden Canpolat

Week	Dates	Work Done in Some Detail	Total Hours Spent
Week 1	24/02/25	Planned semester roadmap and defined task priorities	8
Week 2	3/03/25	Investigated navigation issues; scoped integration with compass/arrow system	14
Week 3	10/03/25	Built quest-to-arrow manager module; linked selected quest to arrow direction	10
Week 4	17/03/25	Improved navigation arrow UI for better visibility and feedback	16
Week 5	24/03/25	Fixed malfunction in Google Places API for quest destination mapping	18
Week 6	07/04/25	Implemented calibration for gyroscope accuracy; tested heading alignment	12
Week 7	14/04/25	Device orientation bug fix; ensured consistent compass readings during movement	18
Week 8	21/04/25	-	0
Week 9	28/04/25	Tested compass stability and orientation accuracy under different circumstances	11
Week 10	5/05/25	Finalized gyroscope-compass integration for arrow navigation	15
Week 11	12/05/25	Reviewed navigation UI polish; adjusted icon scaling and responsiveness	20
Week 12	19/05/25	Coordinated with UI logic for journal-navigation connection; tested directional reliability	20
Week 13	26/05/25	Ran system-level tests on device movement, compass deviation, and tracking feedback	13
Week 14	9/06/25	Performed integration test sweeps; reviewed compass and UI interactions	15
Total Effort in Man-Hours			190
Total Effort in Man-Days			23.75

Notes:

1. This table shows the team member project effort. One Man-Day is Eight Man-Hours.
2. Each team member must fill out the form periodically (preferably at the end of the week of any work done).
3. Each filled-out table must be emailed at the end of each month to a selected Project Member (cc to Project Advisor), who will produce a consolidated table.

COMP 4920 Project Effort Log, Project Code: WQ, 20.06.2025, v1.0
Team Member: Tahsin Berk Öztekin

Week	Dates	Work Done in Some Detail	Total Hours Spent
Week 1	24/02/25	Defined semester objectives; discussed priorities with team	8
Week 2	3/03/25	Investigated Firebase-based build failure in Unity for Android	14
Week 3	10/03/25	Continued debugging build issues; checked Firebase Auth dependencies	10
Week 4	17/03/25	Researched Firebase + Unity build errors; tested fix proposals from online sources	16
Week 5	24/03/25	Explored Unity configuration conflicts; narrowed root cause to Firebase version mismatch	18
Week 6	07/04/25	Verified build issue related to Unity version; aligned Firebase settings accordingly	12
Week 7	14/04/25	Prepared initial DSD v2.0 structure	11
Week 8	21/04/25	-	0
Week 9	28/04/25	Finalized and formatted DSD v2.0 with recent architecture and feature updates	18
Week 10	5/05/25	Conducted pre-documentation review; outlined PM and Final Report structure	15
Week 11	12/05/25	Drafted sections for Product Manual; confirmed UI and build behavior content	20
Week 12	19/05/25	Completed Product Manual; started final document write-up	20
Week 13	26/05/25	Contributed to final report writing and editing	13
Week 14	9/06/25	Proofread all documents; validated formatting	15
Total Effort in Man-Hours			190
Total Effort in Man-Days			23.75

Notes:

1. This table shows the team member project effort. One Man-Day is Eight Man-Hours.
2. Each team member must fill out the form periodically (preferably at the end of the week of any work done).
3. Each filled-out table must be emailed at the end of each month to a selected Project Member (cc to Project Advisor), who will produce a consolidated table.